

Sistemas Distribuídos e Computação em Nuvem

Aula 3 — Concorrência: um servidor para vários clientes

C1 · Fundamentos e comunicação distribuída · 20/08/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — a trilha do semestre

C1 · Fundamentos e comunicação (Aulas 1–7)

Sockets, concorrência, gRPC, REST e **IA como serviço**.

C2 · Coordenação e consistência (Aulas 8–12)

Mensageria, relógios lógicos, **CAP**, **Raft**, resiliência.

C3 · Nuvem, implantação e segurança (Aulas 13–18)

Containers, nuvem, serverless, observabilidade, segurança.

 Você está na **Aula 3** — e o problema de hoje volta o semestre inteiro.

Retomada — o que você fez em casa

 A aula começa consolidando a entrega da Aula 2.

- Você reescreveu o eco em **UDP** (`SOCK_DGRAM`, `sendto` / `recvfrom`) e **mediu** 100 mensagens em TCP × UDP?
- O que os seus números mostraram: **UDP mais rápido**, TCP com o custo das **garantias**?
- Entregou `eco_udp.py` + `medicao.md` ?

 Hoje o seu servidor de eco vai deixar de atender **um** cliente e passar a atender **vários ao mesmo tempo**.

Objetivos desta aula

Ao final, você será capaz de:

1. **Explicar** por que um servidor simples atende **apenas um cliente por vez**.
2. **Usar threads** para atender **vários clientes ao mesmo tempo**.
3. **Identificar e corrigir** uma **condição de corrida** em estado compartilhado.

Conceito 1/3 — O problema: o servidor bloqueante

- Chamadas de rede como `accept()` e `recv()` são **bloqueantes**: param a execução até algo acontecer (uma conexão chegar, um dado ser recebido).
- Num servidor **sequencial**, enquanto ele atende **um** cliente, **todos os outros esperam na fila**.
- Basta **um cliente lento** — ou mal-intencionado — para **travar o atendimento de todos**.
- Para um serviço real, com muitos usuários simultâneos, isso é **inaceitável**.

⚠ É exatamente o seu `servidor_eco.py` da Aula 2: ele faz `accept()` de **um** cliente e só volta a aceitar outro quando o primeiro termina.

Conceito 2/3 — A solução: uma thread por cliente

- **Concorrência:** transformar cada atendimento em uma **thread** — uma linha de execução independente **dentro do mesmo programa**.
- Ao aceitar uma conexão, o servidor **entrega aquele cliente a uma thread própria** e **volta na hora** para o `accept()`, pronto para o próximo.
- Muitos clientes atendidos ao mesmo tempo; um cliente lento atrapalha **só a própria thread**.

Concorrência

Lidar com **várias tarefas em andamento**, revezando o processador.

Paralelismo

Executá-las **literalmente ao mesmo tempo**, em núcleos diferentes.

💡 Para servidores de rede (que passam o tempo **esperando** a rede), threads são ideais: enquanto uma espera, as outras trabalham.

Conceito 3/3 — O preço: condição de corrida

- A concorrência resolve um problema e **cria outro**. Se **duas threads alteram a mesma variável ao mesmo tempo**, o resultado fica **imprevisível** — é a **condição de corrida**.
- **Exemplo clássico (contador)**: duas threads leem **41**, ambas somam 1 e ambas gravam **42** — quando o certo seria **43**. Uma contagem se perdeu.
- Traíçoeiro: o erro aparece **só às vezes**, dependendo do instante em que as threads se cruzam.

💡 Proteção: **exclusão mútua** — só **uma thread por vez** executa o trecho que mexe no dado compartilhado (a **seção crítica**), usando um **lock** (trava). Essa ideia volta o semestre inteiro — entre **réplicas de um banco**, não mais entre threads.

Laboratório

Servidor que atende vários clientes — passo a passo

Lab · Passo 1 — `servidor_multicliente.py`

```
import socket, threading
HOST, PORT = "127.0.0.1", 5000

def atender(conexao, endereco):          # roda em uma THREAD por cliente
    print(f"[servidor] {endereco} conectou", flush=True)
    while True:
        dado = conexao.recv(1024)
        if not dado: break
        conexao.sendall(dado)             # eco
    conexao.close()

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((HOST, PORT)); s.listen()
print(f"[servidor] ouvindo em {HOST}:{PORT}", flush=True)
while True:
    conexao, endereco = s.accept()        # aceita um cliente...
    threading.Thread(target=atender, args=(conexao, endereco),
                      daemon=True).start() # entrega à thread e VOLTA já
```

Lab · Passo 2 — rodar com vários clientes

Terminal 1 — servidor:

```
cd $HOME\Documents\sd-2026-2
python servidor_multicliente.py
# [servidor] ouvindo em 127.0.0.1:5000
```

Terminais 2, 3 e 4 — clientes (abra três!):

```
python cliente.py
> ola do cliente A
eco: ola do cliente A
```

💡 No VS Code, use *Split Terminal* para abrir vários terminais lado a lado. Reaproveite o `cliente_eco.py` da Aula 2 (renomeie para `cliente.py`).

Lab · Passo 3 — o experimento que prova o ganho

Com **três clientes conectados ao mesmo tempo**, observe no terminal do servidor as **três linhas de conexão** — ele fala com todos **em paralelo**.

- Compare com o `servidor_eco.py` da Aula 2 (single-thread): abra dois clientes nele e veja o **segundo travar** até o primeiro sair.
- Agora um cliente lento **não** congela os outros — cada um tem a **sua thread**.

⚠ O laço principal **nunca fica preso** num cliente: ele só faz `accept()` → cria thread → volta ao `accept()`. O trabalho pesado fica nas threads.

O preço aparece — condição de corrida (contador.py)

Duas threads somam **2000 vezes** na **mesma variável**. O `+= 1` é, na real, **ler** → **somar** → **gravar**:

```
import threading, time
total = 0
def soma_muitas():
    global total
    for _ in range(2000):
        atual = total          # LER
        time.sleep(0)         # força a troca de thread aqui
        total = atual + 1      # GRAVAR

ts = [threading.Thread(target=soma_muitas) for _ in range(2)]
[t.start() for t in ts]; [t.join() for t in ts]
print("total:", total)       # deveria ser 4000 — mas sai bem MENOS
```

⚠ Em CPython, o **GIL** costuma *esconder* a corrida no `+= 1`; o `time.sleep(0)` escancara a janela entre **ler** e **gravar**. As duas threads leem o mesmo valor e uma soma se perde — o total despenca (some metade).

A correção — **lock** na seção crítica

```
import threading, time
total = 0
lock = threading.Lock()          # a trava

def soma_muitas():
    global total
    for _ in range(2000):
        with lock:                # SEÇÃO CRÍTICA: uma thread por vez
            atual = total
            time.sleep(0)
            total = atual + 1

ts = [threading.Thread(target=soma_muitas) for _ in range(2)]
[t.start() for t in ts]; [t.join() for t in ts]
print("total:", total)           # agora SEMPRE 4000
```

💡 O `with lock:` garante **exclusão mútua**: enquanto uma thread está dentro, as outras **esperam a vez**.
Custa desempenho — proteja só o **mínimo** necessário.

No seu trabalho — C1.A2

- O seu serviço processa inferências **em segundo plano** com um **worker** — e a concorrência desta aula é o que o **sustenta**.
- A **fila** já vem pronta no kit; a **teoria** dela chega na **Aula 8**.

💡 Puxa direto para as tarefas do kit:

- **TAREFA 3** — o worker **grava o resultado** processado para consulta posterior.
- **TAREFA 5** — **tratamento de erro** no worker (retentativa).

Repositório: `sd-2026-2-kit-c1a2`.

Atividade para casa — teste de carga

1. **Escreva** `carga.py`: dispare **N clientes** (ex.: 10, 50, 100) contra o `servidor_multicliente.py`, cada um enviando algumas mensagens; use **threads** no cliente para lançá-los juntos.
2. **Meça** o tempo total e o tempo médio por cliente conforme **N cresce**.
3. **Compare** com o servidor single-thread da Aula 2 sob a mesma carga.
4. **Escreva** `relatorio_concorrencia.md` com os tempos e a **análise**: onde o single-thread trava e por quê.

 **Entregar até a próxima aula:** `carga.py` + `relatorio_concorrencia.md` com os números que **você** mediu.

Lab · Checkpoints & problemas comuns (Windows)

✓ O que você deve ver

- O servidor imprime **uma linha de conexão por cliente**, sem esperar o anterior sair.
- Cada cliente recebe o **eco** das suas mensagens.
- `contador.py` sai **bem < 4000**; a versão com `lock` sai **= 4000**.

🔧 Se der erro

- `[WinError 10048]` (porta ocupada) → feche servidores antigos ou troque a porta.
- Threads não aparecem “ao mesmo tempo”? Confirme o `daemon=True` e que o `accept()` está **dentro** do laço.
- `Ctrl + C` encerra tudo (threads `daemon` morrem com o principal).

◆ Foco ENADE

O que costuma cair:

- **Concorrência × paralelismo**: a diferença entre os dois.
- **Condição de corrida, exclusão mútua e seção crítica**.
- **Threads e processos**: quando usar cada um.
- **Escalabilidade** de servidores e **gargalos**.

Termos-chave: Thread · Condição de corrida · Exclusão mútua · Seção crítica · Lock · Bloqueante

💡 Questões costumam mostrar um **contador incorreto** ou um cenário de acesso simultâneo e pedir o **diagnóstico** e a **correção**.

Questão de autoavaliação (estilo ENADE)

Duas threads incrementam a **mesma variável sem sincronização** e o total final fica **menor que o esperado**. Qual é o problema e a correção adequada?

- A) *Deadlock*; aumentar o número de threads.
- B) **Condição de corrida**; proteger a seção crítica com exclusão mútua.
- C) Inanição (*starvation*); elevar a prioridade das threads.
- D) Falha de rede; substituir o protocolo de transporte.
- E) *Cold start*; pré-aquecer o serviço.

Resolução — alternativa B

- Acesso concorrente **não sincronizado** a dado compartilhado é uma **condição de corrida**.
- Corrige-se garantindo **exclusão mútua** na **seção crítica**, com um **lock**.
- As demais descrevem problemas **diferentes** (deadlock, starvation, rede, cold start) que não explicam a **perda de contagens**.

💡 É o seu `contador.py` de hoje virando questão de prova.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 7 — Sistemas operacionais e concorrência.
- Documentação do módulo `threading` do Python (`Thread` , `Lock`).
- **Guarde** o `contador.py` : a mesma corrida reaparece entre **réplicas** na Aula 10.

Glossário

- **Thread**: linha de execução independente no mesmo programa.
- **Condição de corrida**: duas threads alteram o mesmo dado ao mesmo tempo.
- **Seção crítica**: trecho que só uma thread pode executar por vez.
- **Lock**: garante exclusão mútua na seção crítica.
- **Bloqueante**: chamada que para a execução até terminar.

Até a próxima aula 🚀

Entregue `carga.py` + `relatorio_concorrencia.md`. A Aula 4 começa revendo isto.

Próxima: Do `RPC` ao `gRPC` — chamada remota moderna, como se fosse uma função local.

 **Voltar ao índice**
todas as aulas